

LOGISPARSE

GUÍA DE ESTUDIO DEFINITIVA

Preparación para la defensa técnica ante el jurado — Concurso de Innovación INACAP

Si puedes explicar una decisión en voz alta, sin notas y sin titubear, la dominas de verdad. Si no puedes, todavía no terminaste de construirla en tu cabeza — y eso es exactamente lo que este documento te va a ayudar a descubrir.

hxcCoder · 2026

ÍNDICE

(Si el índice aparece vacío al abrir el archivo: clic derecho sobre él → Actualizar campo)



CÓMO USAR ESTA GUÍA

Esta guía no es para leer una sola vez: es para usarla en voz alta, simulando la defensa frente al jurado. La técnica viene de Richard Feynman: si no puedes explicar algo en palabras simples, todavía no lo entendiste del todo.

1. Lee el título en negrita y la pregunta que lo acompaña.
2. Sin mirar el código ni tus apuntes, explícalo en voz alta como si estuvieras frente al jurado.
3. Si te trabaste, dudaste, o recurriste a frases como “esto hace algo con...”, anótalo: ahí está el hueco que falta cerrar.
4. Si lo explicaste con seguridad y pudiste justificar el por qué (no solo el qué), marca la casilla ☐.
5. Al cerrar cada sección, completa la caja de autoevaluación con la fecha y tu nivel de confianza.

 *El jurado rara vez pregunta “¿qué hiciste?”. Pregunta “¿por qué lo hiciste así y no de otra forma?”. Prepárate para defender decisiones, no para recitar definiciones.*



SECCIÓN 1 — ARQUITECTURA GENERAL

El “Big Picture”

- ☐ **Razón de ser del proyecto:** ¿Por qué un motor híbrido (Regex + IA) es mejor que solo OCR o solo IA para el mercado chileno?
- ☐ **Flujo completo de datos:** Desde que el usuario sube un PDF hasta que se guarda la corrección en la BD (Ruta: Upload → Validación → OCR → Clasificador → Adaptador → Validación Semántica → Confianza → Estado → Frontend).
- ☐ **Diferencia entre EXTRACTED y NEEDS_REVIEW:** ¿Cuándo se dispara cada uno y por qué el umbral está en 80%?
- ☐ **El ciclo de aprendizaje continuo:** ¿Cómo la tabla data_corrections convierte a LogisParse en un sistema que “mejora solo”?



Autoevaluación — Sección 1

Fecha estudiada: _____ Confianza general (1–5): ☐1 ☐2 ☐3 ☐4 ☐5

¿Podrías defenderlo ante el jurado sin mirar el código? ☐ Sí, sin problema ☐ Necesito repasar

SECCIÓN 2 — BACKEND CORE

FastAPI, Seguridad, DB

Configuración y Entorno (app/core/config.py)

- ☐ **pydantic-settings:** ¿Cómo funciona la carga de variables de entorno y por qué usamos `@lru_cache` en `get_settings`?
- ☐ **ALLOWED_ORIGINS:** ¿Por qué da error si no está configurado y cómo se parsea en el validador?

Base de Datos (app/core/database.py & app/models/)

- ☐ **SQLAlchemy 2.0 Asíncrono:** Diferencia entre `sync` y `async` en SQLAlchemy, y por qué usamos `async_sessionmaker`.
- ☐ **statement_cache_size=0:** El dolor de cabeza con PgBouncer: ¿por qué tuvimos que deshabilitar las prepared statements?
- ☐ **Modelos (BaseModel):** Herencia con Mapped y por qué usamos `str(uuid.uuid4())` en lugar de UUID nativo de PostgreSQL.
- ☐ **Relaciones:** La relación entre Document y DataCorrection (cascade delete). ¿Qué significa `back_populates`?

Migraciones (migrations/ & Alembic)

- ☐ **Alembic:** ¿Cómo se genera una migración (`--autogenerate`) y por qué a veces falla?
- ☐ **La tabla data_corrections:** ¿Por qué esta tabla es el “cerebro” del aprendizaje continuo?

Seguridad (app/core/security.py)

- ☐ **Argon2 vs Bcrypt:** ¿Por qué usamos Argon2 en lugar de bcrypt? ¿Qué error genera si usas la librería equivocada?
- ☐ **JWT (HS256):** ¿Cómo se firma el token, qué contiene el payload (sub) y cómo se valida en `decode_token`?



Autoevaluación — Sección 2

Fecha estudiada: _____ Confianza general (1–5): ☐1 ☐2 ☐3 ☐4 ☐5
 ¿Podrías defenderlo ante el jurado sin mirar el código? ☐ Sí, sin problema ☐ Necesito repasar



SECCIÓN 3 — DOMINIO Y SERVICIOS

El corazón de la extracción

Validación y Limpieza (`app/services/document_extractor.py`)

- ☐ **clean_extracted_text:** ¿Por qué eliminamos líneas con solo números o mayúsculas sostenidas (ej. “CONCEPTO”)?
- ☐ **validate_extracted_data:** La lógica de validación de ciudades chilenas (regex con tildes). ¿Por qué validamos fuerte el origen y destino pero dejamos observaciones como “allow”?
- ☐ **Extracción de texto (pdfplumber vs pytesseract):** ¿Cuándo se usa cada uno y qué pasa si falla la importación (try/except)?

Arquitectura de Adaptadores (`app/services/extractors/`)

- ☐ **Patrón de Diseño “Strategy/Adapter”:** ¿Por qué BaseAdapter es abstracto y cómo obliga a implementar `extract_data` y `calculate_confidence`?
- ☐ **DocumentClassifier:** La lógica de prioridad (Starken vs Guía Chilena). ¿Por qué pusimos la Guía Chilena ANTES que Starken (el error del principio)?
- ☐ **StarkenAdapter:** Sus patrones Regex específicos (PATENTE VEHÍCULO, GUIA N°).
- ☐ **ChileanGuiaAdapter:** ¿Por qué “forzamos” valores como “Puerto Montt” y “Santiago”, y cómo extraemos la patente con `.*`? (regex non-greedy)?
- ☐ **GenericLLMAdapter:** ¿Cómo funciona el parse de OpenAI (Structured Outputs)? ¿Por qué la API key es de Groq pero el cliente es de OpenAI?
- ☐ **El Few-Shot:** ¿Cómo se inyecta el `correction_history` en el prompt de la IA para mejorar resultados futuros?



Autoevaluación — Sección 3

Fecha estudiada: _____ Confianza general (1–5): ☐1 ☐2 ☐3 ☐4 ☐5

¿Podrías defenderlo ante el jurado sin mirar el código? ☐ Sí, sin problema ☐ Necesito repasar



SECCIÓN 4 — PRUEBAS Y CALIDAD

Testing & CI

- ☐ **Pytest Fixtures (conftest.py):** ¿Cómo se sobrescribe get_db para usar SQLite en memoria y por qué es crucial para las pruebas?
- ☐ **Mocking:** El uso de patch para simular extract_document y evitar llamar a OpenAI/OCR en las pruebas.
- ☐ **Cobertura de código (pytest --cov):** ¿Qué mide y por qué un coverage.xml no se sube al repo?
- ☐ **Errores de Ruff:** Diferencia entre E501 (línea larga) e I001 (imports desordenados). ¿Por qué ruff format es necesario?
- ☐ **Mypy (Type Checking):** El plugin pydantic.mypy: ¿por qué tuviste que instalar email-validator y qué son los # type: ignore?



Autoevaluación — Sección 4

Fecha estudiada: _____ Confianza general (1–5): ☐1 ☐2 ☐3 ☐4 ☐5

¿Podrías defenderlo ante el jurado sin mirar el código? ☐ Sí, sin problema ☐ Necesito repasar

SECCIÓN 5 — FRONTEND

Next.js 14 + TypeScript

Estructura y Estilos

- ☐ **Next.js App Router:** La diferencia entre (auth) y (dashboard) como grupos de rutas. ¿Por qué tienen layouts diferentes?
- ☐ **TailwindCSS:** La paleta de colores (Slate, Azul marino) y por qué usamos flex items-center justify-center para centrar los íconos de react-icons.
- ☐ **globals.d.ts:** ¿Por qué tuvimos que declarar módulos CSS para que TypeScript no llorara?

Autenticación (context/AuthContext.tsx)

- ☐ **React Context:** ¿Cómo se provee el user y token a toda la app?
- ☐ **Event Listener de 401:** ¿Cómo el interceptor de Axios dispara un evento 'unauthorized' y el Context lo escucha para hacer logout automático?
- ☐ **Persistencia:** ¿Por qué guardamos el token en localStorage pero el usuario en sessionStorage (o ambos en localStorage)?

Cliente API (lib/api.ts)

- ☐ **Axios Interceptors:** ¿Cómo se inyecta el token en cada request y cómo se manejan los errores 401 globalmente?
- ☐ **Multipart/Form-Data:** ¿Cómo se construye el FormData para la subida de archivos y por qué el Content-Type se define automáticamente?

Componentes Clave

- ☐ **ProtectedRoute:** ¿Cómo verifica el token y redirige a /login si no existe?
- ☐ **Pipeline Visual (UploadDropzone + PipelineVisual):** ¿Cómo se simulan los pasos (Recibido → OCR → IA → Validado) con setTimeout y estados?
- ☐ **CorrectionForm:** ¿Cómo se itera sobre ALL_FIELDS para mostrar todos los campos (incluso vacíos), y cómo se arma el payload para enviar solo los campos modificados?

Tipado Compartido (types/index.ts)

- ☐ **ExtractedLogisticsData:** ¿Por qué tiene extra="allow" y cómo permite que campos como transportista aparezcan en el frontend aunque no estén definidos en el esquema base?



Autoevaluación — Sección 5

Fecha estudiada: _____ Confianza general (1–5): ☐1 ☐2 ☐3 ☐4 ☐5

¿Podrías defenderlo ante el jurado sin mirar el código? ☐ Sí, sin problema ☐ Necesito repasar



SECCIÓN 6 — HERRAMIENTAS DE DESARROLLO

DevOps

- ☐ **Ruff:** ¿Por qué reemplaza a Flake8, Black e Isort a la vez? ¿Cómo se usa ruff check . --fix?
- ☐ **Mypy:** La diferencia entre tipado estático y dinámico, y por qué es útil en Python.
- ☐ **GitHub Actions:** El flujo de CI: ¿qué significa “run-on: ubuntu-latest” y por qué falló al no encontrar pydantic (dependencias)?
- ☐ **Codecov:** ¿Para qué sirve el badge de cobertura?



Autoevaluación — Sección 6

Fecha estudiada: _____ Confianza general (1–5): ☐1 ☐2 ☐3 ☐4 ☐5

¿Podrías defenderlo ante el jurado sin mirar el código? ☐ Sí, sin problema ☐ Necesito repasar



TABLERO DE PROGRESO GENERAL

Completa esta tabla después de repasar cada sección completa. Es tu mapa de qué tan listo estás para la defensa.

Sección	Temas	Fecha	Confianza (1–5)	¿Lista?
 1. ARQUITECTURA GENERAL	4			
 2. BACKEND CORE	10			
 3. DOMINIO Y SERVICIOS	9			
 4. PRUEBAS Y CALIDAD	5			
 5. FRONTEND	12			
 6. HERRAMIENTAS DE DESARROLLO	4			
TOTAL	44			

Plan de Sesiones Sugerido

Agrupar las secciones en bloques de estudio en lugar de intentar todo de una sentada. Escribe tú la fecha de cada sesión:

1. Sesión 1 (Fecha: _____): Secciones 1 y 2 — Arquitectura y Backend Core. Es la base que sostiene todo lo demás.
2. Sesión 2 (Fecha: _____): Sección 3 — Dominio y Servicios. La más densa: dale el tiempo que necesite, sin apurarte.
3. Sesión 3 (Fecha: _____): Secciones 4 y 6 — Testing y DevOps. Temas más cortos, buen combo para una sesión más liviana.
4. Sesión 4 (Fecha: _____): Sección 5 — Frontend.
5. Sesión 5 (Fecha: _____): Repaso cruzado. Elige 5 preguntas al azar de cualquier sección y explícalas sin mirar nada.

Dominar este proyecto no significa memorizar cada línea de código, sino poder explicar — con seguridad y en tus propias palabras — por qué tomaste cada decisión de arquitectura. Esa es la diferencia entre un estudiante que copió un tutorial y un desarrollador que construyó algo real.